

④ 設計判断

テーブル分割

決済メールから家計簿を作るにあたり、テーブルを **3 層に分割**しました。

- **email_messages**: 取得したメールの生データ
- **payment_events**: メールから抽出した決済イベント
- **transactions**: 名寄せ後の実際の支出

分割した理由は、**抽出口ジックや名寄せロジックを後から変更したときに、全件を再計算できるようにするため**です。メールから直接 **transactions** を作る設計にすると、名寄せの閾値を一度調整するだけで Gmail から取り直しになってしまいます。生データを保持することで、この手戻りを避けられます。

リレーション設計

email_messages 1 対 N payment_events 銀行の入出金まとめ通知やカード会社の月次確定明細のように、1 通のメールから複数の決済が読み取れるケースがあるため、1 対多にしました。

transactions 1 対 N payment_events 同じ買い物が EC 側とカード側の両方からメールを発生させるため、複数の決済イベントを 1 つの取引に束ねる必要があります。

中間テーブルを使わなかった判断 当初は **payment_events** と **transactions** を多対多と考え、中間テーブル (**transaction_links**) を置いていました。しかし整理すると、**1 つの決済イベントが属する取引は必ず 1 つ**であり、実際は N 対 1 でした。中間テーブルを廃止し、**payment_events.transaction_id** の外部キーで表現しています。マージ解除は、この列と統合元の **deleted_at / merged_into_id** を戻すことで実現します (下記「削除の設計」)。

transactions が自前のフィールドを持つ判断 「代表となる決済イベントを参照する」設計にすると、名寄せロジックを変更して全件を再計算したときに、ユーザが手で修正した内容が失われてしまいます。そのため **transactions** は金額・日時・店舗・カテゴリを自前で保持し、**is_user_edited** フラグが立った行は再計算の対象外としています。

削除の設計

当初はすべて物理削除の方針でしたが、**transactions** だけを論理削除 (**deleted_at**) に変更しました。

取引を物理削除すると **payment_events.transaction_id** が **SET NULL** になり、決済イベントが「まだ名寄せされていない」状態に戻ります。その結果、**次の名寄せバッチが同じ取引を再生成してしまう** (ユーザが削除した取引が復活する) ことに気づいたためです。論理削除にして行を残せば、その行が決済イベントを保持し続けるため、バッチは処理済みと判断でき、復活しません。

マージの統合元も物理削除しません。 **merged_into_id** に統合先を記録したうえで論理削除します。統合元を物理削除すると、そこに入っていたメモやカテゴリの手修正が失われ、**unmerge** (マージ解除) が成立しないためです。副次的に、**409 ALREADY_MERGED** を「行は存在するが **merged_into_id** が入っている」という状態から判定できるようになりました。物理削除だと行が消えるため **404** としか答えられません。

メール連携の解除も物理削除しません。 **email_messages** は **mail_accounts** に CASCADE でぶら下がっているため、連携解除で行を消すとメールの生データまで失われ、再計算ができなくなります。連携解除は

`status = 'disabled'` への更新とリフレッシュトークンの破棄で表現し、全データの削除は「退会 (`users` の削除)」という別の操作に分けました。

一方、マスタ系は物理削除のままにしています。 `categories` などを論理削除にすると、削除済みの行が `UNIQUE(user_id, name)` を占有し、同じ名前で作り直せなくなるためです。マスタ系の外部キーはすでに `SET NULL` なので、物理削除しても不整合は起きません。

このように **DELETE** の意味はテーブルごとに異なるため、API 仕様書にもエンドポイント単位の対応表を明記しています。

エンドポイントの粒度

アクション型のエンドポイントを分離しました。 `merge / unmerge / classify` は、リソースの状態を書き換えるだけでなく、決済イベントの付け替えや分類ルールの学習といった副作用を伴います。これらを汎用の `PUT /transactions/{id}` に含めると「どのフィールドを送ったか」で挙動が変わり分かりにくくなるため、意図が明示されるアクション型エンドポイントに分けました。

副作用をレスポンスの `meta` で返します。 `PUT /transactions/{id}` でカテゴリを修正すると、その店舗のブランドに対する分類ルールを学習します。学習した内容を `meta.learned_rule` として返すことで、「この修正が今後の取引にも効く」ことを画面上でユーザに伝えられます。学習させたくない場合のために `learn_category` フラグも用意しました。

バッチ処理の API は最小限にしました。メールの取得・解析・名寄せはバックグラウンドのバッチが行うため、API 側は手動同期の受付 (202) と同期履歴の参照のみを提供しています。

インデックス選定

用途を 4 つに分けて選定しました。

1. 画面の主要導線: `transactions (user_id, occurred_at DESC)` — 部分インデックス (`WHERE deleted_at IS NULL`) 取引一覧は「期間で絞って新着順」が基本操作のため。論理削除した行は必ず除外するので、その条件をインデックス側に持たせ、削除件数が増えても索引が太らないようにしています。
2. 集計クエリ: `transactions (user_id, category_id, occurred_at)` 月次サマリのカテゴリ別内訳のため。
3. アルゴリズム専用: `payment_events (user_id, amount_jpy_minor, occurred_at)` 名寄せの候補生成は「金額が一致し、日時が近いイベント」を探す処理なので、そのための複合インデックスを用意しました。画面からは使われませんが、この処理の性能を決める中心的なインデックスです。
4. バッチのキュー取得: `email_messages.parse_status / merchants.geocode_status / payment_events.notified_at` 各工程が「未処理のものだけ」を拾うために使います。該当行が全体のごく一部なので部分インデックスにしています。

加えて、`merchant_aliases (user_id, alias)` はメール 1 通を処理するたびに必ず引くため、最も呼び出し回数が多いインデックスとして優先しました。

その他の判断

- **金額は `bigint` の最小単位**で保持し、浮動小数点を使いません。外貨は「原通貨・原金額・円換算額」の3点で保持します。
- **`payment_events.event_type`** は、名寄せ後にどの金額を採用するか判定と、即時通知を送るかどうかの判定の**両方**に使う中心的な列です。ECの注文確認で通知すると、数日後のカード利用速報で同じ買い物が二重に通知されるためです。
- **`notifications.dedupe_key` に `UNIQUE` 制約**を置き、即時通知・予算超過通知・月次サマリの3種類すべての二重送信を1つの制約で防いでいます。バッチが短い間隔で再実行されるため必須と判断しました。
- **マルチテナント設計**: 当面はシングルユーザでの運用ですが、後から `user_id` を追加するコストが高いため、全業務テーブルに最初から持たせています。
- **認証を2層に分離**: APIを呼ぶための認証 (Bearer トークン) と、メールを読むための認証 (Google OAuth) は目的も有効期限も異なります。後者は暗号化してDBに保持し、APIリクエストには一切使いません。

⑤ まとめ

判断に迷ったこと

1. **`sync_jobs` の粒度**: 即時通知のために短い間隔でバッチを回すため、実行のたびに記録すると1日あたり千件を超える行が増えます。変化がなかった実行は記録しない方針にしましたが、それだけだと「新着がなかった」のか「バッチが止まっていた」のかが区別できなくなります。そこで、**成功時は0件でも `mail_accounts.last_synced_at` を更新すること**にし、死活監視はこの列で行う形にしました。行を増やさずに停止を検知できる一方、実行履歴が歯抜けになるため、障害調査の際に困らないかは相談したい点です。
2. **`merchants` の粒度**: 地図表示は支店単位、集計とカテゴリ規則はブランド単位で行いたいという要求があり、`brands` テーブルを分けるべきか迷いました。今回はテーブル数と実装期間を考慮し、`brand_name` を列として持つ形にしています。
3. **メール本文の保持**: 後からロジックを変えて再解析するには本文が必要ですが、個人の決済情報をそのまま保持することになるため、プライバシーと容量の観点で迷っています。
4. **利用速報と確定明細の金額差**: カードの利用速報と確定明細で金額が変わることがあります。`transactions` に状態列 (`pending / confirmed`) を持たせるか、確からしさの高いイベントで上書きするだけにするかを決めきれていません。